

Dame un texto para estudiar y entender la jerarquía de Chomsky, con ejemplos

La **jerarquía de Chomsky** es una clasificación de las gramáticas formales según qué tan complejas son las reglas que permiten generar un lenguaje. Es un tema central en **teoría de lenguajes formales, autómatas y complejidad computacional**, porque conecta tres ideas: **gramáticas, lenguajes y máquinas capaces de reconocerlos**.

La jerarquía tiene cuatro niveles, numerados de **Tipo 3 a Tipo 0**. Cada nivel puede expresar lenguajes más complejos que el anterior:

$$\text{Tipo 3} \subset \text{Tipo 2} \subset \text{Tipo 1} \subset \text{Tipo 0}$$

Es decir, todo lenguaje regular también es libre de contexto, todo lenguaje libre de contexto también es sensible al contexto, etc. Pero no ocurre necesariamente al revés.

1. Gramáticas Tipo 3: lenguajes regulares

Son las gramáticas más simples. Sus reglas tienen una forma muy restringida, por ejemplo:

$$A \rightarrow aB$$

o

$$A \rightarrow a$$

donde A y B son variables o símbolos no terminales y a es un símbolo terminal.

Estos lenguajes pueden ser reconocidos por un **autómata finito**.

Un ejemplo sencillo es el lenguaje:

$$L = \{a^n b \mid n \geq 0\}$$

Sus cadenas son:

b
ab
aab
aaab
aaaab
...

Una gramática podría ser:

$S \rightarrow aS$
 $S \rightarrow b$

Por ejemplo:

S
→ aS
→ aaS
→ aaaS
→ aaab

Así obtenemos aaab .

Los lenguajes regulares son adecuados para reconocer patrones simples: identificadores, números, ciertos formatos de texto, palabras que terminan en determinada secuencia, etc. Por eso las **expresiones regulares** están relacionadas con este nivel.

2. Gramáticas Tipo 2: lenguajes libres de contexto

Las gramáticas libres de contexto permiten reglas de la forma:

$$A \rightarrow \gamma$$

donde A es un único no terminal y γ puede ser una secuencia de terminales y no terminales.

Aquí aparece algo que los autómatas finitos no pueden manejar fácilmente: **estructuras anidadas o cantidades relacionadas**.

El ejemplo clásico es:

$$L = \{a^n b^n \mid n \geq 0\}$$

Sus cadenas son:

ϵ
ab
aabb
aaabbb
aaaabbbb
...

La cantidad de a debe ser exactamente igual a la cantidad de b .

Una gramática posible es:

$S \rightarrow aSb$
 $S \rightarrow \epsilon$

Para generar:

aaabbbb

hacemos:

S
 $\rightarrow aSb$
 $\rightarrow aaSbb$
 $\rightarrow aaaSbbb$
 $\rightarrow aaabbbb$

¿Por qué este lenguaje no es regular? Porque la máquina necesita **recordar cuántas** a **aparecieron** para luego verificar la misma cantidad de b .

Para hacerlo se utiliza un **autómata con pila**.

Una forma intuitiva de imaginarlo es:

Entrada: aaabbb

a → meter algo en la pila

a → meter algo en la pila

a → meter algo en la pila

b → sacar algo

b → sacar algo

b → sacar algo

Pila vacía → aceptar

Las gramáticas libres de contexto son muy importantes para describir la **sintaxis de lenguajes de programación**.

Por ejemplo:

```
if (condición) {  
    instrucción  
}
```

o expresiones matemáticas anidadas:

$((a + b) * (c - d))$

3. Gramáticas Tipo 1: lenguajes sensibles al contexto

Aquí las reglas pueden depender del contexto que rodea a un símbolo.

Una forma típica de representar una regla es:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

El símbolo A puede transformarse dependiendo de lo que tenga alrededor.

Un ejemplo famoso es:

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Sus cadenas incluyen:

abc
aabbcc
aaabbbccc
aaaabbbbcccc

Aquí tienen que cumplirse simultáneamente:

cantidad de a = cantidad de b = cantidad de c

Esto es más difícil que:

$$a^n b^n$$

porque ahora hay que mantener relacionadas **tres cantidades**.

Este lenguaje no puede reconocerse con un simple autómata con pila.

La máquina asociada a este nivel es el **autómata linealmente acotado** o **LBA — Linear Bounded Automaton**.

Puede imaginarse como una máquina de Turing cuya memoria está limitada proporcionalmente al tamaño de la entrada.

Por ejemplo, para:

aaabbbccc

la máquina podría ir marcando símbolos:

aaabbbccc

Xaabbbccc

XaYbbbccc

XaYbbZcc

XXaYbbZcc

XXaYYbZcc

XXaYYbZZc

XXXYYYZZZ

Cada vez empareja:

- 1 a
- 1 b
- 1 c

Si al final todos están correctamente emparejados, acepta.

4. Gramáticas Tipo 0: lenguajes recursivamente enumerables

Son las gramáticas menos restringidas y, por tanto, las más poderosas.

Sus reglas son básicamente:

$$\alpha \rightarrow \beta$$

con muy pocas restricciones.

La máquina equivalente es la **máquina de Turing**.

Este nivel representa los problemas que pueden ser reconocidos mediante un procedimiento computacional general.

Hay una diferencia importante aquí: una máquina de Turing puede recibir una entrada y:

aceptar
rechazar
o quedarse ejecutando para siempre

Por eso se habla de lenguajes **recursivamente enumerables**.

Un ejemplo conceptual sería un lenguaje que contenga las codificaciones de programas que eventualmente terminan bajo determinadas condiciones.

Este nivel está muy relacionado con conceptos como:

- computabilidad;
- problema de parada;
- decidibilidad;
- máquinas de Turing.

Cómo memorizar la jerarquía

Puedes relacionarla con la cantidad de **memoria** que necesita la máquina.

Tipo	Lenguaje	Máquina	Idea de memoria	Ejemplo
Tipo 3	Regular	Autómata finito	Sin memoria adicional	a^*b
Tipo 2	Libre de contexto	Autómata con pila	Una pila	$a^n b^n$
Tipo 1	Sensible al contexto	Autómata linealmente acotado	Cinta limitada	$a^n b^n c^n$
Tipo 0	Recursivamente enumerable	Máquina de Turing	Memoria general	Cómputos arbitrarios

Una forma especialmente útil de recordarlo es pensar en esta progresión:

Tipo 3

Autómata finito

↓

no necesita contar demasiado

Tipo 2

Autómata con pila

↓

puede recordar cantidades/anidamientos

Tipo 1

LBA

↓

puede mantener varias relaciones

Tipo 0

Máquina de Turing

↓

computación general

Un ejemplo comparativo

Supón que queremos reconocer distintos lenguajes.

Para:

aaaaab

solo necesitamos comprobar:

cero o más a , seguidas de una b .

Esto es **regular**:

$$a^*b$$

Para:

aaaabbbb

necesitamos comprobar:

mismo número de a y b .

Esto requiere memoria:

$$a^n b^n$$

Es **libre de contexto**.

Para:

aaaabbbbcccc

debemos comprobar:

mismo número de a , b y c .

Esto requiere una capacidad aún mayor:

$$a^n b^n c^n$$

Es **sensible al contexto**.

La idea clave es:

A medida que subimos en poder expresivo, la máquina necesita más capacidad para recordar y relacionar información.

Hay una pequeña trampa con la palabra "subir": por número, Tipo 0 es el más potente aunque tenga el número más pequeño. Por eso suele ser más fácil memorizar:

3 → regular
2 → libre de contexto
1 → sensible al contexto
0 → sin restricciones

Y asociarlos así:

3 → Finite Automaton
2 → Pushdown Automaton
1 → Linear Bounded Automaton
0 → Turing Machine

Una pregunta típica de examen sería: **¿Dónde clasificarías $L = \{0^n 1^n \mid n \geq 0\}$?** La respuesta es Tipo 2, libre de contexto, porque necesitas recordar cuántos 0 aparecieron para verificar la misma cantidad de 1, y una pila es suficiente.

Otra sería: **¿Dónde clasificarías $L = \{0^n 1^n 2^n \mid n \geq 1\}$?** Es Tipo 1, sensible al contexto: una sola pila no basta para imponer fácilmente las tres cantidades iguales.

El punto más importante para estudiar la jerarquía de Chomsky no es memorizar únicamente las reglas, sino entender esta cadena:

Gramática $\longleftrightarrow$ Lenguaje $\longleftrightarrow$ Máquina

Por ejemplo:

Gramática libre de contexto $\leftrightarrow a^n b^n \leftrightarrow$ Autómata con pila

Si entiendes esas asociaciones y por qué cada máquina necesita más memoria que la anterior, gran parte del tema se vuelve mucho más intuitiva.



